



MANUEL TECHNIQUE ET D'UTILISATION DU SIMULATEUR

**Navigation autonome par apprentissage par renforcement avec
l'algorithme Soft Actor-Critic (SAC)**

PROJET : Simulateur de navigation autonome pour véhicules guidés automatisés en environnement aéroportuaire.

STRUCTURE D'ACCUEIL : ECAM Louis de Broglie

AUTEURE : Sayan SUOS

ENCADRANTS : Jean-Marc LAFERTÉ
Alain-Jérôme FOUGÈRES

DERNIÈRE MISE À JOUR : 27 août 2026

Table des matières

Introduction	2
1 Installation du simulateur	3
1.1 Prérequis	3
1.2 Installation	3
2 Structure du projet	4
2.1 Organisation des fichiers	4
2.2 Architecture du simulateur	5
2.3 Organisation des classes	6
3 Configuration du simulateur	8
3.1 Configuration des tâches	8
3.2 Configuration des environnements	10
3.3 Configuration des agents	12
3.4 Configuration de la fonction de récompense	14
3.5 Configuration de l'algorithme SAC	15
3.6 Configuration du curriculum d'apprentissage probabiliste	16
4 Utilisation du simulateur	18
4.1 Modes d'exécution	18
4.2 Entraînement d'une politique	19
4.3 Validation d'une politique	22
4.4 Évaluation d'une politique	24
5 Visualisation des résultats	26
5.1 Générer des visualisations	26
5.2 Présenter une démonstration interactive	27
5.3 Génération des métriques, des figures et des animations	28
Répertoire du projet	32
5.4 Répertoire des fichiers sources	32
5.5 Répertoire des dossiers générés	34
5.6 Répertoire des principales classes	34
FAQ	36

Introduction

Le transport de bagages par des véhicules guidés automatisés (AGV) en environnement aéroportuaire soulève des problématiques de navigation autonome, de coordination entre robots et d'évitement d'obstacles dans des environnements dynamiques. Afin d'étudier ces problématiques, un simulateur a été développé pour modéliser différents scénarios de navigation et expérimenter des approches d'apprentissage par renforcement.

Ce simulateur est utilisé dans un contexte général de navigation de robots mobiles autonomes (AMR). Il permet de créer des environnements configurables composés d'un ou plusieurs agents, d'obstacles statiques et d'obstacles dynamiques, puis d'entraîner et d'évaluer des politiques de navigation. La version actuelle repose sur l'algorithme *Soft Actor-Critic* (SAC) et permet d'étudier des scénarios mono-agent et multi-agent, notamment à l'aide d'une politique partagée entre robots.

Ce manuel a pour objectif de présenter l'utilisation du simulateur et d'accompagner l'utilisateur dans sa prise en main. Il décrit successivement les étapes d'installation, l'organisation générale du projet, les fichiers de configuration, les principaux modes d'exécution ainsi que les outils permettant de visualiser et d'analyser les résultats générés.

Ce document se concentre volontairement sur les aspects pratiques liés à l'utilisation du simulateur. Les choix de modélisation, l'algorithme SAC, l'approche multi-agent par *parameter sharing* ainsi que le protocole expérimental sont détaillés dans le document de modélisation associé.

1 Installation du simulateur

Cette section décrit les différentes étapes nécessaires pour installer le simulateur sur une machine locale. Les procédures présentées ont été testées sous macOS et Windows.

1.1 Prérequis

Avant d'installer le simulateur, les logiciels suivants doivent être installés sur la machine :

- Python (version 3.11 ou supérieure) ;
- Git, utilisé pour cloner le dépôt du projet ;
- un environnement virtuel Python.

1.2 Installation

Le projet est distribué sous la forme d'un dépôt Git. La première étape consiste donc à se placer dans le dossier souhaité, puis à cloner le dépôt sur la machine locale :

```
1 git clone https://github.com/sayansuos/baggage-handling-rl.git
2 cd baggage-handling-rl
```

Il est ensuite recommandé de créer un environnement virtuel afin d'isoler les dépendances du projet. Celui-ci peut ensuite être activé selon le système d'exploitation utilisé :

- Sous macOS ou Linux :

```
1 python -m venv .venv
2 source ./venv/bin/activate
```

- Sous Windows :

```
1 python -m venv .venv
2 .\venv\Scripts\activate
```

Une fois l'environnement virtuel activé, les dépendances du projet peuvent être installées à partir du fichier `requirements.txt` :

```
1 python -m pip install -r requirements.txt
```

Une fois cette étape terminée, le simulateur est prêt à être utilisé.

2 Structure du projet

Cette section présente l'organisation du projet et l'architecture du simulateur, en décrivant sa structure, son fonctionnement général ainsi que les principales classes qui le composent.

2.1 Organisation des fichiers

Le projet est organisé en plusieurs dossiers, chacun regroupant un ensemble de fonctionnalités spécifiques. Le rôle des principaux fichiers et dossiers est résumé dans le Tableau 1.

TABLE 1 – Description des principaux dossiers et fichiers du projet.

Élément	Description
configs/	Fichiers de configuration du simulateur, regroupant les paramètres des environnements, des agents, des fonctions de récompense, des tâches d'entraînement et d'évaluation, de l'algorithme SAC et du curriculum.
docs/	Documentation du projet.
figures/	Figures, graphiques et animations générées lors de l'entraînement ou de l'évaluation du modèle.
logs/	Métriques générées lors de l'entraînement et de l'évaluation du modèle.
rl/	Implémentation des algorithmes d'apprentissage par renforcement.
simulator/	Cœur du simulateur. Regroupe les principaux composants de la simulation (environnement, agents, obstacles), les outils nécessaires à l'apprentissage par renforcement (observations, fonction de récompense), la gestion des interactions entre les entités ainsi que le calcul des trajectoires des obstacles dynamiques par A*.
utils/	Fonctions utilitaires utilisées par l'ensemble du projet (visualisation, journalisation, chargement des configurations, etc.).
cli.py	Définit l'interface en ligne de commande du projet, notamment les sous-commandes disponibles et leurs différents paramètres.
main.py	Point d'entrée du projet. Charge les tâches et les arguments fournis en ligne de commande, puis appelle la fonction d'exécution correspondant au mode sélectionné.
runner.py	Regroupe les fonctions responsables de l'orchestration des entraînements, des validations, des évaluations, des démonstrations et de la génération des animations.

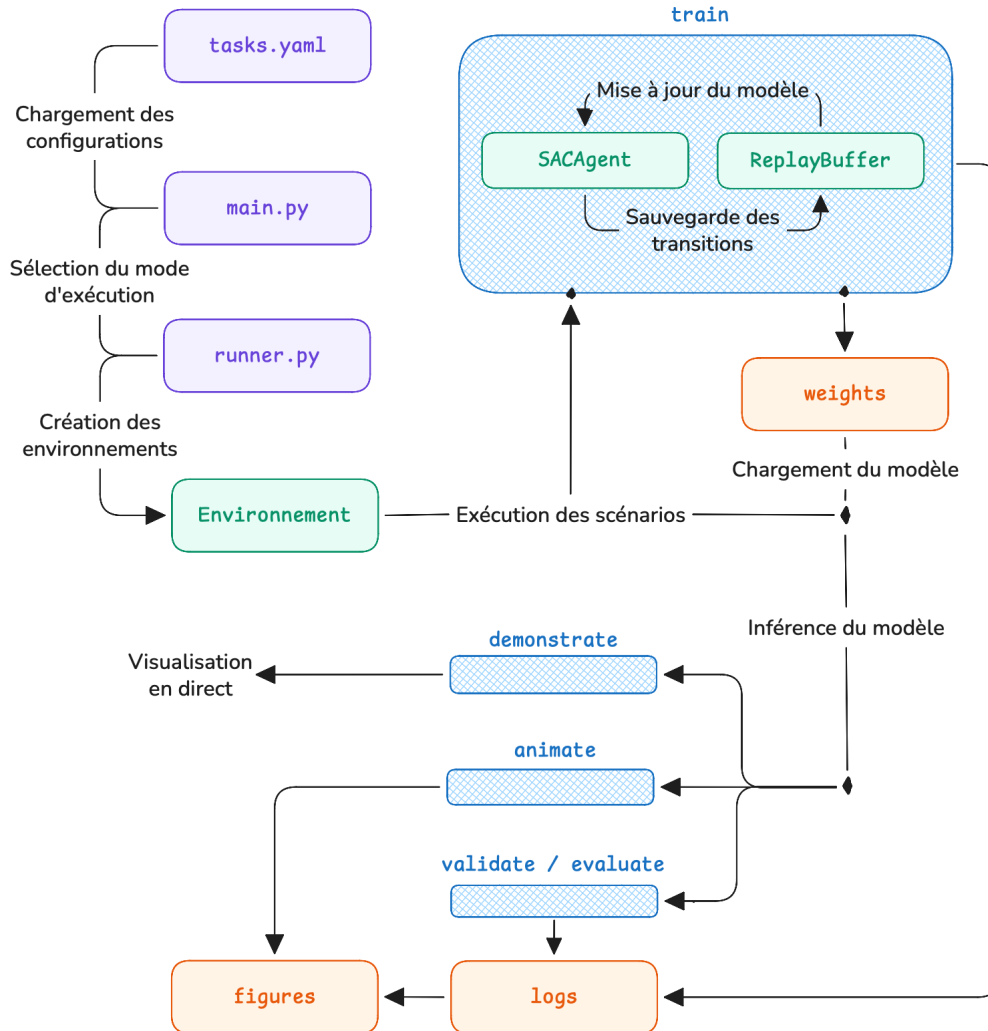
Remarque

Le dossier `simulator/` contient l'ensemble des composants nécessaires au fonctionnement du simulateur. Dans la plupart des cas, l'utilisateur n'a pas besoin de modifier ce dossier : la personnalisation du comportement du simulateur s'effectue principalement à travers les fichiers de configuration présentés dans la Section 3.

2.2 Architecture du simulateur

La Figure 1 présente l'architecture fonctionnelle du simulateur ainsi que les principales étapes d'exécution d'une tâche.

FIGURE 1 – Architecture fonctionnelle du simulateur.



Quel que soit le mode d'exécution sélectionné, le programme débute dans le fichier `main.py`. Celui-ci récupère les arguments fournis en ligne de commande, charge les tâches et les configurations nécessaires, puis appelle les fonctions correspondantes du fichier `runner.py`.

En mode **train**, une politique de navigation est entraînée à l'aide de l'algorithme *Soft Actor-Critic* (SAC), dans un contexte mono-agent ou multi-agent avec *parameter sharing*. L'apprentissage est réalisé sur une liste de tâches préconfigurées, selon un curriculum séquentiel ou probabiliste. Les transitions générées au cours de l'exécution permettent de mettre à jour progressivement les réseaux de neurones du modèle, tandis que les poids de la politique et les métriques d'apprentissage sont régulièrement enregistrés.

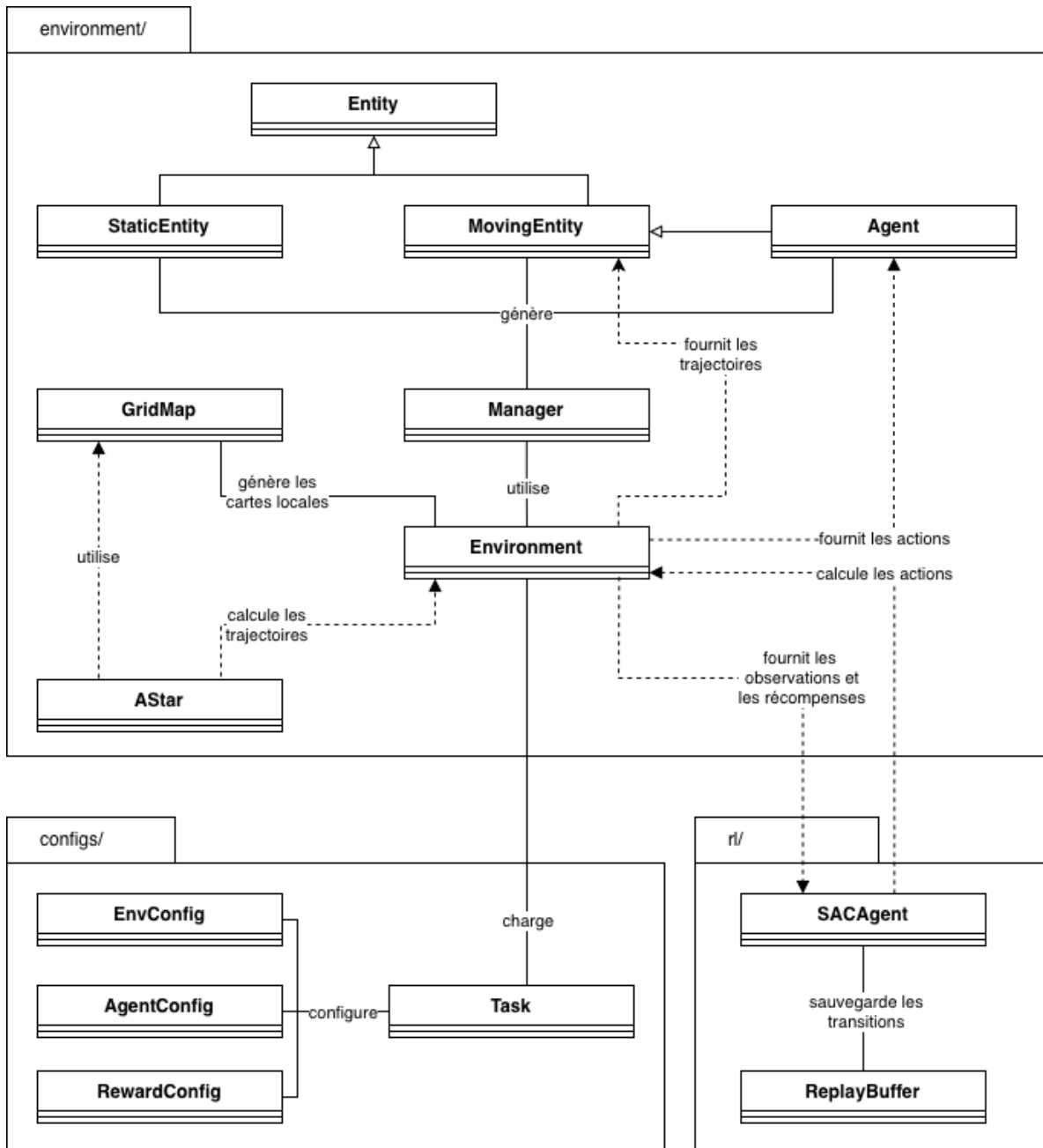
Les modes **validate**, **evaluate**, **animate** et **demo** reposent quant à eux sur une politique déjà entraînée, dont les poids sont chargés avant l'exécution. Selon le mode sélectionné, le simulateur permet ensuite d'évaluer quantitativement la politique ou d'en observer le comportement.

Le fonctionnement détaillé de chaque mode d'exécution est présenté dans la Section 4.

2.3 Organisation des classes

Le simulateur repose sur une architecture orientée objet (POO) dans laquelle les différents éléments présents dans un environnement sont représentés par des classes. La Figure 2 présente un diagramme de classes simplifié des principaux composants du simulateur.

FIGURE 2 – Diagramme de classes simplifié du simulateur.



La classe `Entity` constitue la classe de base des objets présents dans l'environnement. Elle définit les propriétés communes aux différentes entités, notamment leur position, leurs dimensions ainsi que les mécanismes de détection des collisions.

Deux classes héritent de celle-ci : la classe `StaticEntity` qui représente les obstacles statiques de l'environnement, ainsi que la classe `MovingEntity` qui représente les entités capables de se déplacer, telles que des obstacles dynamiques. La classe `Agent` hérite de cette dernière et complète son comportement avec les propriétés nécessaires à la navigation et à l'apprentissage par renforcement.

La classe `Environment` regroupe les agents, les obstacles statiques et les obstacles dynamiques qui composent un scénario. Elle est responsable de leur initialisation (via la classe `Manager`), de l'application des actions et trajectoires, de la mise à jour de la simulation, du calcul des observations et des récompenses, ainsi que de la détection de la fin d'un épisode.

La classe `GridMap` fournit une représentation discrète de l'environnement. Elle est utilisée à la fois pour construire les cartes locales observées par les agents et pour calculer les trajectoires des obstacles dynamiques à l'aide de la classe `AStar`.

Enfin, l'apprentissage par renforcement est assuré par la classe `SACAgent` qui s'appuie sur un *replay buffer*, implémenté dans la classe `ReplayBuffer`), ainsi que sur les réseaux de neurones de l'extracteur de caractéristiques, de l'*actor* et du *critic*, qui sont respectivement représentés par les classes `FeatureExtractor`, `ActorNetwork` et `CriticNetwork`. Les paramètres des environnements, des agents et de la fonction de récompense sont quant à eux définis respectivement par les classes `EnvConfig`, `AgentConfig` et `RewardConfig`.

Remarque

Le diagramme de classes présenté est volontairement simplifié. Les attributs et les méthodes des différentes classes n'y sont pas représentés afin de privilégier une vue d'ensemble de l'architecture du simulateur et des principales relations entre ses composants.

3 Configuration du simulateur

La configuration du simulateur repose principalement sur un ensemble de fichiers YAML, regroupés dans le dossier `configs/`. Ces fichiers permettent de définir les caractéristiques des environnements, des agents, de la fonction de récompense ainsi que les tâches d’entraînement et d’évaluation. Les hyperparamètres propres à l’algorithme SAC et au curriculum d’apprentissage sont quant à eux regroupés dans des dataclasses définies dans le fichier `config.py`.

TABLE 2 – Description des fichiers de configuration.

Élément	Description
<code>config.py</code>	Définit les différentes classes de configuration (dataclasses) utilisées pour représenter les paramètres du simulateur.
<code>tasks.yaml</code>	Définit les tâches à exécuter.
<code>environments.yaml</code>	Définit les environnements.
<code>agents.yaml</code>	Définit les paramètres des agents.
<code>rewards.yaml</code>	Définit les paramètres de la fonction de récompense.

Remarque

Les fichiers de configuration YAML sont automatiquement chargés par le module `config_loader`, qui instancie les dataclasses `EnvConfig`, `AgentConfig`, `RewardConfig` et `Task`.

Les paramètres de l’algorithme SAC et du curriculum d’apprentissage sont définis directement dans le fichier `config.py` via les dataclasses `SACConfig` et `Curriculum`. Ils sont utilisés au début de l’entraînement par les fonctions d’exécution et les modules associés à l’algorithme.

Les sections suivantes détaillent le contenu de chaque type de configuration.

3.1 Configuration des tâches

Les tâches à exécuter sont définies dans différentes sections du fichier `tasks.yaml`. Chaque section est identifiée par un nom choisi par l’utilisateur et contient une liste de tâches partageant un même objectif, par exemple l’entraînement ou l’évaluation. Le nom de la section à utiliser est ensuite précisé lors de l’exécution. Chaque tâche correspond à un scénario et associe un environnement, une configuration d’agent ainsi qu’une fonction de récompense.

Par exemple, une section d’entraînement peut être définie sous le nom « `train_v1` ».

```
1 train_v1:
2   - name: task1
3     env_config: task1
```

```

4   agent_config: agent_1
5   reward_config: reward_1
6   n_steps: 100_000

```

De même, une section dédiée à l'évaluation peut être définie sous le nom « eval ».

```

1  eval:
2    - name: eval1
3      env_config: eval1
4      agent_config: agent_1
5      reward_config: reward_1
6      n_steps: null

```

TABLE 3 – Description des paramètres de Task.

<i>Paramètre</i>	<i>Description</i>
name	Nom de la tâche.
env_config	Nom de la configuration d'environnement.
agent_config	Nom de la configuration de l'agent.
reward_config	Nom de la fonction de récompense.
n_steps	Nombre d'étapes d'entraînement.

Les tâches peuvent être enchaînées selon différentes stratégies de curriculum d'apprentissage.

- Dans une approche séquentielle, elles sont exécutées dans l'ordre où elles apparaissent, les poids obtenus à l'issue d'une tâche étant réutilisés pour initialiser la suivante.
- Dans l'approche probabiliste, les tâches sont sélectionnées dynamiquement selon une distribution de probabilité qui privilégie la tâche actuellement ciblée tout en conservant une probabilité non nulle de sélectionner les autres tâches.

Dans les deux cas, les différentes configurations permettent d'augmenter progressivement la complexité des scénarios rencontrés au cours de l'apprentissage.

Remarque

Dans le cas d'un curriculum séquentiel, chaque tâche d'entraînement est exécutée pendant un nombre d'étapes défini (n_steps). Dans le cas du curriculum probabiliste, l'entraînement est réparti en blocs de taille fixe, et les tâches sont sélectionnées dynamiquement selon la distribution de probabilité associée.

Les tâches d'évaluation n'utilisent pas le paramètre n_steps, elles sont exécutées sur un nombre fixe d'épisodes.

3.2 Configuration des environnements

Les environnements sont définis dans le fichier `environments.yaml`. Chaque environnement décrit un scénario de simulation en précisant les dimensions de la carte, les modes de génération des entités ainsi que les paramètres contrôlant la création des obstacles et le déroulement d'un épisode.

Chaque configuration d'environnement est définie dans une section et est identifiée par un nom unique, qui peut ensuite être référencé dans `tasks.yaml` à l'aide du paramètre `env_config`.

```
1  train_v1:
2    - name: task1
3      width: 64
4      height: 32
5
6      agent_mode: random
7      env_mode: random
8
9      nb_agents: 1
10     nb_static_obstacles: 4
11     nb_moving_obstacles: 2
12     nb_targets: 2
13
14     margin: 3
15     max_attempts: 100
16     max_steps: 300
17
18     thickness: 1
19     radius_min: 0.5
20     radius_max: 1.0
```

TABLE 4 – Description des paramètres de EnvConfig.

Paramètre	Description
width	Largeur de l'environnement.
height	Hauteur de l'environnement.
agent_mode	Définit le mode de génération des agents et de leurs objectifs.
env_mode	Définit le type d'environnement et la disposition des obstacles statiques.
nb_agents	Nombre d'agents présents dans le scénario.
nb_static_obstacles	Nombre d'obstacles statiques générés, lorsque le mode d'environnement le permet.
nb_moving_obstacles	Nombre d'obstacles dynamiques générés.
nb_targets	Nombre de cibles successives attribuées aux agents et obstacle dynamiques.
margin	Distance minimale imposée entre les entités lors de leur génération.
max_attempts	Nombre maximal de tentatives pour générer une position valide avant l'abandon.

<i>Paramètre</i>	<i>Description</i>
max_steps	Nombre maximal d'étapes autorisées pour un épisode.
thickness	Épaisseur des murs délimitant l'environnement.
radius_min	Rayon minimal des obstacles dynamiques générés aléatoirement.
radius_max	Rayon maximal des obstacles dynamiques générés aléatoirement.

Les paramètres `env_mode` et `agent_mode` déterminent la stratégie de génération des scénarios. Selon les valeurs choisies, les positions des agents, des obstacles statiques, des obstacles dynamiques et des cibles sont générées suivant des règles différentes, permettant de construire des environnements adaptés à différents niveaux de difficulté. Les règles de génération associées à ces modes sont implémentées dans la classe `Manager`.

TABLE 5 – Description des modes de génération de `env_mode`.

<i>Paramètre</i>	<i>Description</i>
random	Génère aléatoirement les positions et les dimensions des obstacles statiques.
warehouse	Génère une disposition prédéfinie d'obstacles statiques représentant des allées d'entrepôt.
crossing	Génère les obstacles statiques autour du centre de l'environnement selon une disposition circulaire.
fixed_n	Génère les obstacles statiques sur n colonnes fixes, de dimensions déterministes.
fixed_random_n	Génère les obstacles statiques sur n colonnes fixes, de dimensions aléatoires.
hospital	Génère un environnement structuré représentant un couloir principal desservant plusieurs salles, avec une zone d'ascenseur située à une extrémité.
airport	Génère un environnement aéroportuaire ouvert dans lequel des obstacles statiques peuvent être ajoutés.

TABLE 6 – Description des modes de génération de `agent_mode`.

<i>Paramètre</i>	<i>Description</i>
random	Génère aléatoirement les positions initiales et les cibles des agents et des obstacles dynamiques, en respectant les contraintes de distance minimale.
fixed_n	Génère les positions initiales et les cibles des agents de part et d'autre des n colonnes d'obstacles statiques. Les obstacles dynamiques sont placés dans la partie supérieure ou inférieure de l'environnement, avec des cibles situées dans la zone opposée.

<i>Paramètre</i>	<i>Description</i>
crossing	Génère les positions initiales des agents et des obstacles dynamiques sur un cercle, puis leur attribue une cible opposée afin de créer un scénario de croisement.
hospital	Génère les agents dans la zone d'ascenseur avec des cibles situées dans les salles, puis un retour final à leur position initiale. Les obstacles dynamiques circulent dans le couloir principal.
airport	Génère les agents à des positions initiales aléatoires avec des objectifs alternant entre les zones de ramassage et de dépôt, puis un retour final à leur position initiale. Pour chaque agent, un point de ramassage et un point de dépôt sont sélectionnés parmi les trois disponibles et restent identiques pendant toute la simulation. Les obstacles dynamiques évoluent entre ces deux zones.

Selon le mode de génération sélectionné, certains paramètres sont interprétés différemment. Dans les modes `warehouse`, `fixed_n`, `fixed_random_n` et `crossing`, la disposition et les dimensions des obstacles sont déterminées automatiquement à partir des caractéristiques du scénario. Le mode `hospital` génère la structure du couloir, de l'ascenseur et des salles, dont le nombre dépend du nombre d'obstacles demandé. Dans le mode `airport`, les éventuels obstacles statiques supplémentaires sont générés aléatoirement avec une taille fixe.

Remarque

Les différents modes de génération ont été conçus pour produire des scénarios favorables à l'apprentissage par renforcement. Les positions des agents, des obstacles et des cibles suivent alors des règles permettant de reproduire les situations recherchées (croisement, franchissement d'obstacles, etc.) afin de favoriser des interactions pertinentes, tout en garantissant une diversité suffisante des scénarios rencontrés.

3.3 Configuration des agents

Les caractéristiques des agents sont définies dans le fichier `agents.yaml`. Cette configuration regroupe les paramètres physiques du robot ainsi que les paramètres définissant son espace des observations et son espace des actions.

Chaque configuration d'agent est définie par une entrée de la liste `agents`, identifiée par un nom unique, qui peut ensuite être référencé dans le fichier `tasks.yaml` à l'aide du paramètre `agent_config`.

```

1  agents:
2    - name: agent_1
3
4      n_maps: 3
5
6      v_min: 0.0
```

```

7     v_max: 2.0
8     omega_min: -1.05
9     omega_max: 1.05
10
11    radius: 0.5
12    length_view: 15
13    length_view_decrease: null
14
15    collision_threshold: 0.5
16    reach_threshold: 0.25

```

TABLE 7 – Description des paramètres de AgentConfig.

Paramètre	Description
n_maps	Nombre de cartes locales successives conservées dans l’observation afin d’intégrer une information temporelle.
v_min	Vitesse linéaire minimale autorisée pour l’agent.
v_max	Vitesse linéaire maximale autorisée pour l’agent.
omega_min	Vitesse angulaire minimale autorisée pour l’agent.
omega_max	Vitesse angulaire maximale autorisée pour l’agent.
radius	Rayon de l’agent utilisé pour la détection des collisions.
length_view	Taille de la carte locale observée par l’agent.
length_view_decrease	Facteur permettant de réduire la taille de la carte locale en fonction de la vitesse de l’agent. Une valeur null désactive cette adaptation.
collision_threshold	Distance à partir de laquelle l’agent est considéré en collision avec un obstacle ou un autre agent.
reach_threshold	Distance maximale entre l’agent et sa cible pour considérer que celle-ci est atteinte.

Les paramètres `v_min`, `v_max`, `omega_min` et `omega_max` définissent directement les bornes de l’espace des actions de l’agent. Les commandes calculées par l’algorithme d’apprentissage sont automatiquement ramenées dans ces intervalles avant d’être appliquées au simulateur.

Remarque

Les paramètres `n_maps` et `length_view` contrôlent la quantité d’informations disponible pour l’agent. Une carte locale plus grande ou un nombre plus important de cartes successives représentent mieux l’environnement et son évolution, mais complexifient l’apprentissage.

3.4 Configuration de la fonction de récompense

Les paramètres de la fonction de récompense sont définis dans le fichier `rewards.yaml`. Cette configuration permet d’ajuster l’importance des différentes composantes de la récompense utilisées durant l’apprentissage, explicitées dans le document de modélisation.

Chaque configuration de récompense est définie par une entrée de la liste `rewards`, et est identifiée par un nom unique, qui peut ensuite être référencé dans le fichier `tasks.yaml` à l’aide du paramètre `reward_config`.

```
1  rewards:
2    - name: reward_1
3
4      beta1: 5.0
5      beta2: 0.5
6      beta3: 1.0
7
8      linear_bonus_factor: 2.0
9      progress_reduction_factor: 0.2
10     angular_malus_factor: -2.0
11     safety_malus_factor: -1.5
12
13     goal_bonus: 400.0
14     step_malus: -2.0
15     collision_malus: -200.0
16
17     safety_threshold: 3.0
```

TABLE 8 – Description des paramètres de RewardConfig.

Paramètre	Description
beta1	Pondération du terme lié à la progression vers la cible.
beta2	Pondération du terme lié à la rotation de l’agent.
beta3	Pondération du terme lié au respect des distances de sécurité.
linear_bonus_factor	Coefficient bonus proportionnel à la vitesse linéaire de l’agent.
progress_reduction_factor	Facteur réduisant la récompense de progression lorsque l’agent se trouve à proximité d’un obstacle.
angular_malus_factor	Pénalité appliquée en fonction de la vitesse angulaire de l’agent.
safety_malus_factor	Pénalité appliquée lorsque l’agent évolue à une distance inférieure au seuil de sécurité.
goal_bonus	Récompense attribuée lorsque l’agent atteint sa cible.
step_malus	Pénalité appliquée à chaque étape afin d’encourager des trajectoires plus courtes.

<i>Paramètre</i>	<i>Description</i>
<code>collision_malus</code>	Pénalité appliquée lorsqu’une collision est détectée.
<code>safety_threshold</code>	Distance à partir de laquelle la pénalité de sécurité est appliquée.

Les paramètres de cette configuration permettent d’ajuster l’importance relative des différentes composantes de la récompense. En modifiant les coefficients de pondération ou les bonus et pénalités associés à certains comportements, il est possible d’orienter l’apprentissage vers des politiques privilégiant, par exemple, une navigation plus rapide, plus prudente ou plus fluide.

Remarque

La fonction de récompense influence directement le comportement appris par l’agent. Une modification importante de ces paramètres modifie l’objectif d’optimisation et nécessite généralement un nouvel entraînement de la politique.

3.5 Configuration de l’algorithme SAC

Les principaux hyperparamètres de l’algorithme Soft Actor-Critic sont regroupés dans la dataclass `SACConfig`, définie dans le fichier `config.py`. Contrairement aux configurations précédentes, ces paramètres ne sont pas chargés depuis un fichier YAML mais directement définis dans le code de la dataclass.

```

1  @dataclass
2  class SACConfig:
3      obs_size: int = 7
4      tau: float = 0.005
5      alpha: float = 0.15
6      batch_size: int = 64
7      critic_lr: float = 3e-4
8      actor_lr: float = 3e-5
9      gamma: float = 0.99
10     reparam_noise: float = 1e-6
11     feature_size: int = 64
12     hidden_size: int = 128
13     mem_size: int = 100_000

```

TABLE 9 – Description des paramètres de `SACConfig`.

<i>Paramètre</i>	<i>Description</i>
<code>obs_size</code>	Taille du vecteur d’observation, hors carte locale.
<code>tau</code>	Coefficient utilisé pour la mise à jour progressive des réseaux cibles.

<i>Paramètre</i>	<i>Description</i>
alpha	Coefficient contrôlant l'importance du terme d'entropie dans SAC.
batch_size	Nombre de transitions utilisées pour chaque mise à jour des réseaux.
critic_lr	Taux d'apprentissage des critic networks.
actor_lr	Taux d'apprentissage de l'actor network.
gamma	Facteur d'actualisation des récompenses futures.
reparam_noise	Petite constante utilisée lors de l'échantillonnage des actions afin d'améliorer la stabilité numérique.
feature_size	Dimension du vecteur latent représentant l'état de l'agent.
hidden_size	Dimension des couches cachées des réseaux actor et critic.
mem_size	Taille maximale du replay buffer.

Remarque

Certains paramètres comme `obs_size`, `feature_size` ou `hidden_size` dépendent principalement de la représentation de l'état et de l'architecture des réseaux. D'autres, tels que les taux d'apprentissage, `alpha`, `gamma` ou `batch_size`, peuvent être ajustés afin d'améliorer les performances ou la stabilité de l'entraînement.

3.6 Configuration du curriculum d'apprentissage probabiliste

Les paramètres contrôlant le curriculum probabiliste sont regroupés dans la dataclass `Curriculum`, également définie dans le fichier `config.py`. Cette configuration détermine notamment la durée des blocs d'entraînement et les conditions utilisées pour faire évoluer la tâche privilégiée.

```

1 @dataclass
2 class Curriculum:
3     chunk_steps: int = 10_000
4     n_chunks: int = 5
5     threshold: float = 0.9
6     n_eval_episodes: int = 100

```

TABLE 10 – Description des paramètres de `Curriculum`.

<i>Paramètre</i>	<i>Description</i>
chunk_steps	Nombre d'étapes d'entraînement effectuées pour chaque bloc.
n_chunks	Nombre de blocs d'entraînement réalisés entre deux phases d'évaluation.

<i>Paramètre</i>	<i>Description</i>
threshold	Seuil de taux de réussite utilisé pour déterminer si une tâche peut être considérée comme maîtrisée.
n_eval_episodes	Nombre d'épisodes utilisés pour évaluer la politique sur chaque tâche.

Le curriculum probabiliste consiste à sélectionner successivement des tâches d'entraînement selon une distribution de probabilité favorisant une tâche courante, tout en conservant une probabilité non nulle de sélectionner les autres tâches. Après plusieurs blocs d'entraînement, la politique est évaluée afin de déterminer si la tâche privilégiée peut être considérée comme maîtrisée et si le curriculum peut progresser vers une tâche plus complexe. Le fonctionnement de cette stratégie est détaillé dans le document de modélisation.

Remarque

Les paramètres de Curriculum influencent directement l'organisation de l'apprentissage, notamment la durée passée sur chaque tâche et la fréquence des phases d'évaluation. Cette configuration n'est toutefois utilisée que lorsque le curriculum probabiliste est activé.

4 Utilisation du simulateur

Cette section décrit les principaux modes d'exécution du simulateur ainsi que la procédure permettant d'entraîner, de valider et d'évaluer une politique d'apprentissage par renforcement. Elle présente également le déroulement général d'une expérience, depuis son lancement jusqu'à la production des résultats.

4.1 Modes d'exécution

Le simulateur propose plusieurs modes d'exécution permettant notamment d'entraîner une nouvelle politique, d'évaluer une politique existante ou de générer des visualisations. Le fichier `main.py` constitue le point d'entrée du programme. Le mode d'exécution est sélectionné directement depuis la ligne de commande, en précisant la sous-commande correspondante lors du lancement du programme.

TABLE 11 – Description des modes d'exécution.

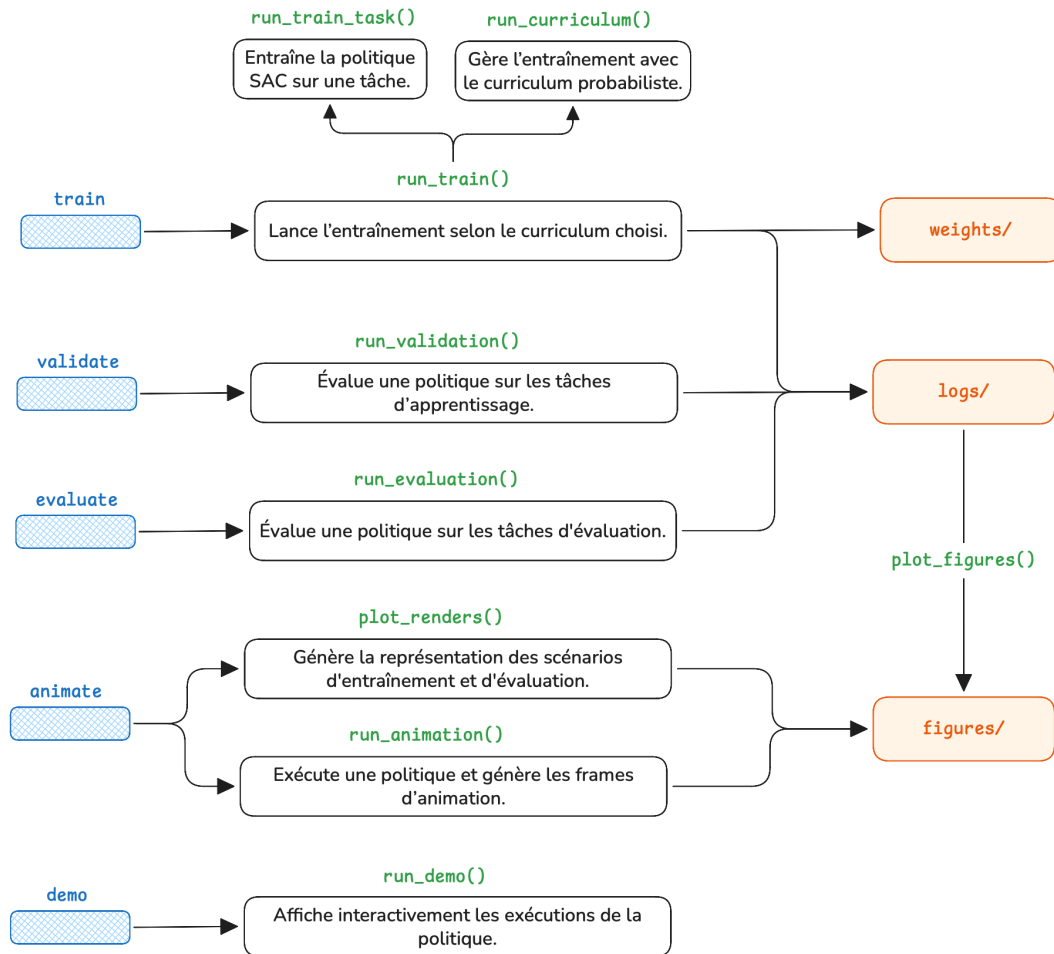
<i>Sous-commande</i>	<i>Description</i>
<code>train</code>	Entraîne une politique SAC sur une liste de tâches. Selon la stratégie choisie, les tâches peuvent être exécutées séquentiellement ou sélectionnées selon un curriculum probabiliste.
<code>validate</code>	Évalue une politique sur les tâches utilisées durant l'apprentissage afin de vérifier les performances obtenues au cours du curriculum.
<code>evaluate</code>	Évalue une politique sur une liste de tâches dédiées à l'évaluation afin d'étudier ses performances finales et sa capacité de généralisation à des configurations différentes de celles rencontrées durant l'apprentissage.
<code>animate</code>	Génère des animations de l'exécution d'une politique entraînée (GIF et MP4), ainsi que des représentations des configurations d'environnement correspondantes.
<code>demo</code>	Charge les animations générées et permet de visualiser interactivement le comportement de la politique.

```
1 python main.py <sous-commande> [options]
```

Plus particulièrement, la sous-commande détermine le mode d'exécution du simulateur, tandis que les options permettent d'en ajuster le comportement selon les besoins de l'utilisateur. Les sous-commandes disponibles ainsi que leurs paramètres sont définis dans le fichier `cli.py`, et présentées dans le Tableau 11 ci-dessus. Les arguments fournis sont ensuite interprétés par `main.py`, qui charge les configurations nécessaires puis appelle les fonctions correspondantes du fichier `runner.py`.

La Figure 3 résume le déroulement classique d'une expérience, depuis la configuration des scénarios jusqu'à l'exploitation des résultats générés.

FIGURE 3 – Workflow d’exécution et organisation des résultats générés par le simulateur.



Les modes **train**, **validate** et **evaluate** sont détaillés dans la suite de cette section. Les modes **animate** et **demo** sont quant à eux présentés dans la section 5.

4.2 Entraînement d’une politique

La sous-commande **train** permet d’entraîner une politique de navigation à l’aide de l’algorithme SAC. L’entraînement est réalisé sur une section de tâches définie dans le fichier `tasks.yaml` et suit un curriculum séquentiel ou probabiliste. Selon le nombre d’agents sélectionnés pour l’apprentissage, la politique peut être entraînée dans un contexte mono-agent ou multi-agent avec l’approche de *parameter sharing*.

```
1 python main.py train [options]
```

Lancement de l’entraînement

L’entraînement est lancé depuis la racine du projet. Plusieurs options permettent de sélectionner les tâches à utiliser, de choisir la stratégie du curriculum, de définir le nombre d’agents participant à l’apprentissage ou encore d’initialiser l’entraînement à partir d’une politique existante. Elles sont présentées dans le Tableau 12 ci-dessous.

TABLE 12 – Options de la sous-commande `train`.

<i>Option</i>	<i>Description</i>
<code>--task-section</code>	Nom de la section de tâches à utiliser dans le fichier <code>tasks.yaml</code> . Cette option est obligatoire.
<code>--policy-name</code>	Nom attribué à la politique entraînée. Cette option est obligatoire.
<code>--start-task</code>	Indice de la première tâche sélectionnée dans la section. Par défaut : 0.
<code>--end-task</code>	Indice de la dernière tâche sélectionnée dans la section. Par défaut : None.
<code>--n-trained-agents</code>	Nombre d'agents dont les transitions participent à l'apprentissage de la politique partagée. Par défaut : 1.
<code>--init-policy-name</code>	Nom d'une politique existante utilisée pour initialiser l'entraînement. Par défaut : None.
<code>--init-checkpoint-name</code>	Nom du checkpoint de la politique existante utilisé pour initialiser l'entraînement. Par défaut : None.
<code>--max-steps</code>	Fixe un nombre maximal d'étapes pour l'entraînement. Par défaut : None.
<code>--no-max-steps</code>	Désactive la limite globale du nombre d'étapes d'entraînement.
<code>--sequential-curriculum</code>	Active l'utilisation du curriculum séquentiel.

Remarque

Les options `--init-policy-name` et `--init-checkpoint-name` doivent être renseignées conjointement. Lorsqu'elles ne sont pas précisées, une nouvelle politique est initialisée pour l'entraînement.

Par exemple, la commande suivante entraîne une politique nommée "policy_v1" sur les tâches de la section "train_v1" :

```
python main.py train --task-section "train_v1" --policy-name "policy_v1"
```

Le curriculum séquentiel peut être activé à l'aide de l'option `--sequential-curriculum` :

```
python main.py train --task-section "train_v1" --policy-name "policy_v1"
↪ --sequential-curriculum
```

Pour faire participer plusieurs agents à l'apprentissage de la politique partagée, le nombre d'agents peut être précisé à l'aide de l'option `--n-trained-agents`. Par exemple :

```
1 python main.py train --task-section "train_v1" --policy-name "policy_v1"
   ↪ --n-trained-agents 2
```

Remarque

Le nombre d'agents présents dans un scénario est défini par le paramètre `nb_agents`, tandis que l'option `--n-trained-agents` détermine combien d'entre eux participent directement à l'apprentissage. Dans le cadre du parameter sharing, les transitions de ces agents alimentent un même *replay buffer* et contribuent ainsi à l'apprentissage d'une politique SAC commune.

Il est également possible d'initialiser un nouvel entraînement à partir d'un checkpoint existant. Le nom de la politique d'origine et celui du checkpoint doivent alors être précisés conjointement :

```
1 python main.py train --task-section "train_v1" --policy-name "policy_v2" \
2   --init-policy-name "policy_v1" --init-checkpoint-name "final"
```

Fonctionnement de l'entraînement

Lors de l'exécution de la sous-commande `train`, le fichier `main.py` charge la section de tâches sélectionnée, en appliquant éventuellement les bornes `--start-task` et `--end-task`. L'ensemble des paramètres est ensuite transmis à la fonction `run_train()` du fichier `runner.py`, qui assure l'orchestration générale de l'entraînement en fonction de la stratégie de curriculum choisie.

- Dans le cas du curriculum séquentiel, les tâches sont parcourues dans l'ordre où elles apparaissent dans la section de tâches sélectionnée. La même politique SAC est entraînée progressivement sur chacune d'entre elles, de manière à transférer les connaissances acquises d'une tâche vers la suivante.
- Dans le cas du curriculum probabiliste, l'entraînement est divisé en blocs successifs. Les tâches sont sélectionnées selon une distribution de probabilité qui favorise la tâche actuellement ciblée. Après plusieurs blocs, la politique est évaluée sur l'ensemble des tâches afin de déterminer si le curriculum peut progresser vers une tâche plus complexe. Le fonctionnement détaillé de cette stratégie est présenté dans le document de modélisation.

Pour chaque tâche sélectionnée, un environnement est généré selon les configurations associées. Les agents qui participent à l'apprentissage utilisent la politique SAC de manière stochastique et leurs transitions sont enregistrées dans le *replay buffer* partagé. Ces transitions sont ensuite échantillonnées afin de mettre à jour les réseaux de neurones du modèle. Les autres agents présents dans l'environnement utilisent la politique courante mais n'alimentent pas directement le *replay buffer*.

Les obstacles dynamiques ne sont pas contrôlés par la politique SAC mais suivent des trajectoires calculées par l'*algorithme A** au début de l'épisode. À chaque étape, ils se déplacent d'un mètre, selon les quatre directions de la grille : haut, bas, gauche et droite. Lorsqu'ils atteignent leur cible finale, ils parcourent la trajectoire en sens inverse et effectuent ainsi des aller-retours jusqu'à la fin

de l'épisode.

Sauvegarde du modèle

Au cours de l'entraînement, différents checkpoints de la politique sont sauvegardés dans le dossier `rl/sac/weights/`. Chaque politique possède un dossier identifié par son nom, à l'intérieur duquel les différents checkpoints sont enregistrés. Le nom de la politique et celui du checkpoint permettent ensuite de sélectionner précisément les poids à charger, notamment à l'aide des options `-init-policy-name` et `-init-checkpoint-name`.

- Dans le cas d'un curriculum séquentiel, un checkpoint `<task_name>` est sauvegardé à l'issue de chaque tâche afin de conserver les différentes étapes de l'apprentissage. Un checkpoint final correspond à l'état de la politique à la fin du curriculum.
- Dans le cas du curriculum probabiliste, le checkpoint `current` correspond au dernier état sauvegardé de la politique en cours d'apprentissage et permet notamment de reprendre un entraînement interrompu. Le checkpoint `"best"` conserve la politique ayant obtenu les meilleures performances au cours des phases d'évaluation, tandis que le checkpoint `"final"` correspond à l'état de la politique à la fin de l'entraînement.

Les métriques produites durant l'apprentissage sont également enregistrées. L'organisation des fichiers générés ainsi que les outils permettant de les analyser et de les visualiser sont présentés dans la Section 5.

4.3 Validation d'une politique

La sous-commande `validate` évalue une ou plusieurs politiques sur les tâches d'entraînement sélectionnées. Elle permet ainsi de mesurer les performances obtenues sur les scénarios rencontrés au cours de l'apprentissage.

```
1 python main.py validate [options]
```

Lancement de la validation

La validation est lancée depuis la racine du projet. Plusieurs options permettent de sélectionner les tâches et la politique à utiliser, de choisir les checkpoints à charger et de définir le nombre d'épisodes exécutés ou rendus pour chaque tâche. Elles sont présentées dans le Tableau 13 ci-dessous.

TABLE 13 – Options de la sous-commande `validate`.

<i>Option</i>	<i>Description</i>
<code>--task-section</code>	Nom de la section de tâches à utiliser dans le fichier <code>tasks.yaml</code> . Cette option est obligatoire.
<code>--policy-name</code>	Nom de la politique à valider. Cette option est obligatoire.

<i>Option</i>	<i>Description</i>
<code>--start-task</code>	Indice de la première tâche sélectionnée dans la section. Par défaut : 0.
<code>--end-task</code>	Indice de la dernière tâche sélectionnée dans la section. Par défaut : None.
<code>--checkpoint-strategy</code>	Stratégie utilisée pour sélectionner le checkpoint à charger pour chaque tâche, soit "matching", "final", "best" ou "current". Par défaut : "final".
<code>--n-episodes</code>	Nombre d'épisodes exécutés pour chaque tâche. Par défaut : 500.
<code>--n-renders</code>	Nombre d'épisodes rendus pour chaque tâche. Par défaut : 5.

Remarque

La stratégie "matching" ne peut être utilisée qu'avec une politique entraînée selon un curriculum séquentiel, pour laquelle des checkpoints intermédiaires ont été sauvegardés à l'issue de chaque tâche. Les stratégies "best" et "current" sont quant à elles destinées aux politiques entraînées avec le curriculum probabiliste. La stratégie "final" peut être utilisée dans les deux cas.

Par exemple, la commande suivante valide le checkpoint "final" de la politique "policy_v1" sur les quatre premières tâches de la section "train_v1", à partir de 200 épisodes par tâche, sans générer de rendu :

```
1 python main.py validate --task-section "train_v1" --policy-name "policy_v1" \
2   --start-task 0 --end-task 3 \
3   --n-episodes 200 --n-renders 0
```

Fonctionnement de la validation

Lors de l'exécution de la sous-commande `validate`, le fichier `main.py` charge la section de tâches sélectionnée, en appliquant éventuellement les bornes `--start-task` et `--end-task`. L'ensemble des paramètres est ensuite transmis à la fonction `run_validation()` du fichier `runner.py`.

Pour chaque tâche, la politique est chargée selon la stratégie définie par `--checkpoint-strategy`.

- La stratégie "final" permet d'évaluer l'état de la politique à la fin de l'entraînement, tandis que "current" permet d'évaluer le dernier état enregistré.
- La stratégie "best" utilise le checkpoint ayant obtenu les meilleures performances au cours de l'entraînement et est principalement adaptée au curriculum probabiliste.
- Enfin, la stratégie "matching" est principalement utilisée avec un curriculum séquentiel : pour chaque tâche, elle charge le checkpoint sauvegardé à l'issue de cette même tâche, ce qui permet d'examiner les performances obtenues aux différentes étapes du curriculum.

La politique est ensuite exécutée sur le nombre d'épisodes défini par `-n-episodes`. Les actions sont générées à partir de la politique entraînée, de manière déterministe sans mise à jour des réseaux de neurones ni ajout de transitions au *replay buffer*. Les métriques obtenues permettent notamment d'évaluer le taux de réussite, le taux de collision et le temps de trajet sur chacune des tâches.

Lorsque `-n-renders` est non nul, un nombre limité d'épisodes est également rendu afin de permettre une analyse visuelle du comportement de la politique. Les métriques et visualisations produites au cours de la validation sont enregistrées pour être exploitées ensuite.

Remarque

La validation est conçue pour être réalisée sur les tâches utilisées durant l'entraînement. La fonction ne vérifie toutefois pas la nature de la section fournie correspond : il appartient à l'utilisateur de sélectionner une section de tâches adaptée. L'utilisation d'autres tâches est techniquement possible, mais correspond alors davantage à une évaluation de la politique.

4.4 Évaluation d'une politique

La sous-commande `evaluate` évalue une ou plusieurs politiques sur les tâches d'évaluation sélectionnées. Contrairement à la validation, qui mesure les performances sur les scénarios utilisés durant l'apprentissage, l'évaluation vise à étudier les performances finales de la politique et sa capacité de généralisation à des configurations différentes de celles rencontrées au cours de l'entraînement.

```
1 python main.py evaluate [options]
```

Lancement de l'évaluation

L'évaluation est lancée depuis la racine du projet. Plusieurs options permettent de sélectionner les tâches et la politique à utiliser, de choisir le checkpoint à charger et de définir le nombre d'épisodes exécutés ou rendus pour chaque tâche. Elles sont présentées dans le Tableau 14 ci-dessous.

```
1 python main.py evaluate [options]
```

TABLE 14 – Options de la sous-commande `evaluate`.

<i>Option</i>	<i>Description</i>
<code>--task-section</code>	Nom de la section de tâches à utiliser dans le fichier <code>tasks.yaml</code> . Cette option est obligatoire.
<code>--policy-name</code>	Nom de la politique à évaluer. Cette option est obligatoire.
<code>--start-task</code>	Indice de la première tâche sélectionnée dans la section. Par défaut : <code>0</code> .

<i>Option</i>	<i>Description</i>
<code>--end-task</code>	Indice de la dernière tâche sélectionnée dans la section. Par défaut : <code>None</code> .
<code>--checkpoint-name</code>	Nom du checkpoint de la politique à évaluer. Par défaut : <code>"final"</code> .
<code>--n-episodes</code>	Nombre d'épisodes exécutés pour chaque tâche. Par défaut : <code>500</code> .
<code>--n-renders</code>	Nombre d'épisodes rendus pour chaque tâche. Par défaut : <code>5</code> .

Par exemple, la commande suivante évalue la politique `"policy_v1"`, à partir de son checkpoint `"final"`, sur les quatre premières tâches de la section `"eval"`, à partir de 200 épisodes par tâche, sans générer de rendu :

```
1 python main.py evaluate --task-section "eval" --policy-name "policy_v1" \
2   --start-task 0 --end-task 3 \
3   --n-episodes 200 --n-renders 0
```

Fonctionnement de l'évaluation

Lors de l'exécution de la sous-commande `evaluate`, le fichier `main.py` charge la section de tâches sélectionnée, en appliquant éventuellement les bornes `--start-task` et `--end-task`. L'ensemble des paramètres est ensuite transmis à la fonction `run_evaluation()` du fichier `runner.py`.

La même politique est chargée pour évaluer l'ensemble des tâches sélectionnées. Pour chaque tâche, elle est exécutée sur le nombre d'épisodes défini par `-n-episodes`. Les actions sont générées à partir de la politique entraînée, de manière déterministe sans mise à jour des réseaux de neurones ni ajout de transitions au *replay buffer*. Les métriques obtenues permettent notamment d'évaluer le taux de réussite, le taux de collision et le temps de trajet sur chacune des tâches.

Lorsque `-n-renders` est non nul, un nombre limité d'épisodes est également rendu afin de permettre une analyse visuelle du comportement de la politique. Les métriques et visualisations produites au cours de l'évaluation sont enregistrées pour être exploitées ensuite.

Remarque

L'évaluation est conçue pour être réalisée sur des tâches différentes de celles utilisées durant l'entraînement. La fonction ne vérifie toutefois pas la nature de la section fournie : il appartient à l'utilisateur de sélectionner une section de tâches adaptée. L'utilisation des tâches d'apprentissage est techniquement possible, mais correspond alors davantage à une validation de la politique.

5 Visualisation des résultats

Cette section présente les différents outils permettant de visualiser les résultats produits par le simulateur. Elle décrit les fonctions permettant d'observer l'exécution d'une politique en direct, de générer des animations, de représenter les scénarios ainsi que les principaux fichiers générés.

5.1 Générer des visualisations

La sous-commande `animate` permet de générer des visualisations associées à une politique entraînée sur une section de tâches donnée. Elle permet notamment de produire des représentations des environnements ainsi que des animations illustrant le comportement de la politique au cours de son exécution.

```
1 python main.py animate [options]
```

Lancement de l'animation

La génération des visualisation est lancée depuis la racine du projet. Plusieurs options permettent de sélectionner les tâches et la politique à utiliser, de choisir le checkpoint à charger et de définir la fréquence d'affichage des animations. Elles sont présentées dans le Tableau 15 ci-dessous.

TABLE 15 – Options de la sous-commande `animate`.

<i>Option</i>	<i>Description</i>
<code>--task-section</code>	Nom de la section de tâches à utiliser dans le fichier <code>tasks.yaml</code> . Cette option est obligatoire.
<code>--policy-name</code>	Nom de la politique à animer. Cette option est obligatoire.
<code>--checkpoint-name</code>	Nom du checkpoint de la politique à utiliser. Par défaut : <code>"final"</code> .
<code>--fps</code>	Nombre d'images par seconde à afficher dans les animations. Par défaut : <code>10</code> .

Par exemple, la commande suivante génère les visualisations de la politique `"policy_v1"`, à partir de son checkpoint `"best"`, sur les tâches de la section `"eval"`, avec une fréquence de 10 images par seconde :

```
1 python main.py animate --task-section "eval" --policy-name "policy_v1" \  
2     --checkpoint-name "best"
```

Fonctionnement de l'animation

Lors de l'exécution de la sous-commande `animate`, le fichier `main.py` charge la section de tâches sélectionnée. Les fonctions de visualisation sont ensuite appelées afin de représenter les environnements et d'exécuter la politique sur les différentes tâches.

- La fonction `plot_renders()`, du fichier `utils/plotting.py`, génère une représentation de la configuration initiale des tâches sélectionnées et rassemble les différents scénarios au sein d'une même figure. Elle permet d'avoir une vision globale des environnements utilisés.
- La fonction `run_animation()`, du fichier `runner.py`, exécute la politique de manière déterministe sur les tâches sélectionnées et conserve les états successifs de l'environnement sous forme d'images. Ces images sont ensuite assemblées afin de produire une animation aux formats GIF et MP4.

Les visualisations générées sont enregistrées afin de pouvoir être consultées ultérieurement. Leur organisation dans les dossiers de résultats est détaillée à la fin de cette section.

5.2 Présenter une démonstration interactive

La sous-commande `demo` permet de visualiser interactivement une animation précédemment générée à partir d'une politique entraînée. Elle est principalement destinée à l'analyse qualitative du comportement de la politique.

```
1 python main.py demo [options]
```

Lancement de la démonstration

La démonstration est lancée depuis la racine du projet. Plusieurs options permettent de sélectionner les tâches et la politique à utiliser, ainsi que de choisir le checkpoint à charger. Elles sont présentées dans le Tableau 16 ci-dessous.

TABLE 16 – Options de la sous-commande `demo`.

<i>Option</i>	<i>Description</i>
<code>--task-section</code>	Nom de la section de tâches à utiliser dans le fichier <code>tasks.yaml</code> . Cette option est obligatoire.
<code>--policy-name</code>	Nom de la politique à visualiser. Cette option est obligatoire.
<code>--checkpoint-name</code>	Nom du checkpoint de la politique à utiliser. Par défaut : "final".

Par exemple, la commande suivante lance une démonstration de la politique "policy_v1", à partir de son checkpoint "best", sur les tâches de la section "eval" :

```
1 python main.py demo --task-section "eval" --policy-name "policy_v1" \
2   --checkpoint-name "best"
```

Fonctionnement de la démonstration

Lors de l'exécution de la sous-commande `demo`, le fichier `main.py` charge les paramètres et les transmet à la fonction `run_demo()` du fichier `runner.py`. Celle-ci charge les animations déjà générées

pour les tâches sélectionnées et les affiche dans une interface interactive. L'utilisateur peut mettre la lecture en pause ou la reprendre à l'aide de la barre d'espace et parcourir manuellement les différentes images à l'aide des flèches gauche et droite.

Remarque

Le mode **demo** ne génère pas lui-même les animations. Les animations correspondantes doivent donc avoir été créées au préalable à l'aide de la sous-commande **animate**, avec la même section de tâches, la même politique et le même checkpoint. Si les fichiers attendus ne sont pas disponibles, la démonstration ne peut pas être lancée correctement.

5.3 Génération des métriques, des figures et des animations

Les différents modes d'exécution génèrent automatiquement plusieurs fichiers permettant de suivre l'apprentissage, d'évaluer les performances des politiques entraînées et de conserver les visualisations produites. Ces résultats sont principalement répartis entre les dossiers `logs/`, `figures/` et `rl/sac/weights/`.

L'organisation des fichiers dépend du mode d'exécution, de la politique utilisée et, lorsque cela est pertinent, du checkpoint chargé. Dans la suite, `<policy>` désigne le nom de la politique, `<checkpoint>` le checkpoint utilisé, `<task>` le nom d'une tâche et `<section>` le nom d'une section de tâches définie dans `tasks.yaml`.

Métriques et journaux d'exécution

Les métriques produites par les modes **train**, **validate** et **evaluate** sont enregistrées dans le dossier `logs/`, organisé selon ces trois modes d'exécution.

- Pendant l'entraînement, les résultats sont regroupés dans le dossier : `logs/train/<policy>/`. Les métriques sont cumulées : soit au fil des tâches successives dans le cas du curriculum séquentiel, ou au fil des différents blocs dans le cas du curriculum probabiliste.
- Pendant la validation et l'évaluation, les résultats dépendent du checkpoint utilisé et sont respectivement enregistrés dans les dossiers `logs/validation/<policy>/<checkpoint>/` et `logs/evaluation/<policy>/<checkpoint>/`.

Pour la validation, la valeur de `<checkpoint>` dépend du paramètre `--checkpoint-strategy` en entrée. Les stratégies `"current"`, `"final"` et `"best"` utilisent respectivement les dossiers correspondants pour toutes les tâches sélectionnées. Avec la stratégie `"matching"`, chaque tâche est évaluée à partir du checkpoint portant son propre nom.

Dans chaque dossier, les fichiers présentés dans le Tableau 17 sont générés. Dans le cas des modes **validate** et **evaluate**, chaque fichier correspond à une tâche et le nom est précédé du préfixe `<task>_`.

TABLE 17 – Fichiers générés dans le dossier logs/.

<i>Fichier</i>	<i>Description</i>
<code>metrics.csv</code>	Contient les principales métriques enregistrées au cours des épisodes, notamment le retour cumulé, le taux de réussite, le taux de collision, le temps de trajet et les vitesses moyennes.
<code>rewards.csv</code>	Contient les différentes composantes de la fonction de récompense.
<code>debug.csv</code>	Contient des informations détaillées sur le déroulement de l’entraînement, notamment les positions, les vitesses, les distances aux obstacles et l’état des agents à chaque étape durant l’épisode.

Remarque

Durant le mode **train**, les informations du fichier `debug.csv` ne sont enregistrées que périodiquement afin de limiter le volume de données générées. Cette périodicité est paramétrée dans la fonction `run_sac()` du module `rl/sac/sac.py`.

Figures et animations

Les métriques produites au cours de l’entraînement, de la validation et de l’évaluation sont également utilisées pour générer automatiquement différentes figures. Celles-ci sont enregistrées dans le dossier `figures/` selon une organisation similaire à celle des journaux d’exécution.

- Les figures d’entraînement sont regroupées dans le dossier : `figures/train/<policy>/`.
- Les figures de validation et d’évaluation sont associées au checkpoint utilisé et respectivement enregistrées dans les dossiers `figures/validation/<policy>/<checkpoint>/` et `figures/evaluation/<policy>/<checkpoint>/`.

La nature des fichiers diffère selon le mode d’exécution : ceux produits durant l’entraînement sont présentés dans le Tableau 18, tandis que ceux produits durant la validation et l’évaluation sont présentés dans le Tableau 19.

TABLE 18 – Fichiers générés dans le dossier `figures/` en mode **train**.

<i>Fichier</i>	<i>Description</i>
<code>performances.png</code>	Représente l’évolution des principales métriques de performance au cours de l’entraînement, notamment les taux de réussite, de collision et de dépassement de durée maximale.
<code>rewards.png</code>	Représente l’évolution du retour cumulé et des différentes composantes de la fonction de récompense.

<i>Fichier</i>	<i>Description</i>
velocities.png	Représente l'évolution des vitesses linéaires et angulaires moyennes au cours de l'entraînement.

TABLE 19 – Fichiers générés dans le dossier figures/ en mode **validate** ou **evaluate**.

<i>Fichier</i>	<i>Description</i>
performances.png	Représente un résumé des métriques de performance pour chaque tâche de la section utilisée, notamment les taux de réussite, de collision et de dépassement de durée maximale.
rewards.png	Représente le retour cumulé moyen pour chaque tâche de la section utilisée.
velocities.png	Représente les vitesses linéaires et angulaires moyennes pour chaque tâche.
<task>_anim.gif et <task>_anim.mp4	Animations générées à partir des épisodes rendus.

Le mode **animate** permet de générer des visualisations indépendamment des phases d'entraînement, de validation ou d'évaluation. Les fichiers produits sont regroupés dans un dossier associé à la politique et au checkpoint choisis, par exemple : figures/demo/<policy>/<checkpoint>/. Une représentation de la configuration type de l'environnement pour chaque tâche d'une section est enregistrée sous la forme <section>_renders.png, tandis que l'exécution de la politique sur l'ensemble des tâches d'une section est sauvegardée sous forme d'animation, notamment dans les fichiers <section>_anim.gif et <section>_anim.mp4.

Sauvegarde des politiques entraînées

Les poids des réseaux de neurones sont enregistrés dans le dossier : rl/sac/weights/. Chaque politique possède un sous-dossier qui correspond à son nom, et les différents états sauvegardés sont ensuite distingués par des checkpoints : rl/sac/weights/<policy>/<checkpoint>/.

Chaque checkpoint contient les paramètres du modèle, répartis dans les fichiers présentés dans le Tableau 20.

TABLE 20 – Fichiers générés dans le dossier rl/sac/weights/.

<i>Fichier</i>	<i>Description</i>
actor.pt	Poids du réseau ActorNetwork.
critic_1.pt	Poids du premier réseau critique CriticNetwork.
critic_2.pt	Poids du second réseau critique CriticNetwork.

<i>Fichier</i>	<i>Description</i>
arget_critic_1.pt	Poids du premier réseau critique cible CriticNetwork.
target_critic_2.pt	Poids du second réseau critique cible CriticNetwork.

Ces poids permettent de recharger exactement un état donné de la politique pour poursuivre un entraînement ou pour l'utiliser lors d'une validation, d'une évaluation ou d'une visualisation.

- Dans le cas d'un curriculum séquentiel, des checkpoints intermédiaires portant le nom des différentes tâches sont conservés afin de retrouver l'état de la politique à l'issue de chacune des étapes du curriculum. Un checkpoint "final" correspond à la politique obtenue à la fin de l'entraînement.
- Dans le cas du curriculum probabiliste, trois checkpoints particuliers peuvent être conservés. Le checkpoint "current" correspond au dernier état sauvegardé de la politique en cours d'apprentissage et permet notamment de reprendre un entraînement interrompu. Le checkpoint "best" conserve l'état ayant obtenu le meilleur taux de réussite global au cours du curriculum. Enfin, le checkpoint "final" correspond à l'état de la politique à la fin de l'entraînement.

La séparation entre le nom de la politique et celui du checkpoint permet de sélectionner précisément les poids à utiliser. Cette distinction est notamment utilisée par les options `--init-policy-name` et `--init-checkpoint-name` lors de l'entraînement, `--checkpoint-strategy` lors de la validation, ainsi que `--checkpoint-name` pour l'évaluation et la génération des visualisations.

Répertoire du projet

Cette section regroupe les principaux éléments constituant le projet afin de faciliter la navigation dans le code. Elle complète la présentation générale de l’architecture, en Section 2, en proposant un répertoire détaillé des fichiers sources, des dossiers générés et des principales classes utilisées par le simulateur.

5.4 Répertoire des fichiers sources

Les principaux fichiers sources du projet sont présentés dans le Tableau 21 ci-dessous. Pour chacun d’eux, le tableau précise son rôle ainsi que les principales classes ou fonctions qu’il contient.

TABLE 21 – Répertoire des fichiers sources.

<i>Fichier</i>	<i>Rôle</i>	<i>Principaux éléments</i>
./configs/		
config.py	Définit les structures de configuration utilisées par le simulateur et l’algorithme d’apprentissage.	Task, EnvConfig, AgentConfig, RewardConfig, SACConfig, Curriculum
tasks.yaml	Définit les différentes sections de tâches utilisées pour l’entraînement, la validation et l’évaluation.	
environments.yaml	Définit les configurations des environnements et des scénarios.	
agents.yaml	Définit les caractéristiques physiques et les espaces d’observation et d’action des agents.	
rewards.yaml	Définit les paramètres des fonctions de récompense.	
./rl/sac/		
agent.py	Implémente l’agent SAC et les principales opérations d’apprentissage et d’inférence	SACAgent
networks.py	Définit les réseaux de neurones utilisés par SAC.	FeatureExtractor, ActorNetwork, CriticNetwork
memory.py	Implémente le <i>replay buffer</i> utilisé pendant l’apprentissage.	ReplayBuffer
sac.py	Regroupe les fonctions assurant l’entraînement et l’évaluation de SAC sur une tâche.	run_sac(), evaluate_sac()
./simulator/		

<i>Fichier</i>	<i>Rôle</i>	<i>Principaux éléments</i>
<code>geometry.py</code>	Regroupe les fonctions géométriques communes au simulateur.	
<code>spaces.py</code>	Définit les espaces d'observation et d'action.	
<code>./.../entities/</code>		
<code>entity.py</code>	Définit la classe de base commune aux entités du simulateur.	<code>Entity</code>
<code>static_entity.py</code>	Définit les entités statiques telles que les murs et obstacles fixes.	<code>StaticEntity</code>
<code>moving_entity.py</code>	Définit les entités mobiles, notamment les obstacles dynamiques.	<code>MovingEntity</code>
<code>agent.py</code>	Définit les robots mobiles contrôlés par la politique.	<code>Agent</code>
<code>node.py</code>	Définit les nœuds utilisés pour la recherche de chemin.	<code>Node</code>
<code>./.../environment/</code>		
<code>environment.py</code>	Implémente l'environnement de simulation et les interactions entre les entités.	<code>Environment</code>
<code>manager.py</code>	Gère la génération et l'initialisation des différents scénarios.	<code>Manager</code>
<code>gridmap.py</code>	Construit la représentation discrète de l'environnement.	<code>GridMap</code>
<code>pathfinding.py</code>	Implémente le calcul des trajectoires des obstacles dynamiques.	<code>AStar</code>
<code>rewards.py</code>	Implémente le calcul de la fonction de récompense.	
<code>terminations.py</code>	Gère les conditions de terminaison et de troncature des épisodes.	
<code>./utils/</code>		
<code>config_loader.py</code>	Charge les fichiers YAML et construit les objets de configuration correspondants.	<code>load_tasks()</code>
<code>logging.py</code>	Assure l'enregistrement des métriques et informations de débogage.	<code>log()</code>
<code>plotting.py</code>	Génère les graphiques, représentations et animations.	<code>plot_animation()</code> , <code>plot_renders()</code> , <code>plot_figures()</code>
<code>./</code>		
<code>cli.py</code>	Définit les sous-commandes et options disponibles depuis la ligne de commande.	<code>build_parser()</code> , <code>parse_args()</code>

<i>Fichier</i>	<i>Rôle</i>	<i>Principaux éléments</i>
<code>main.py</code>	Point d'entrée du programme. Interprète la commande et prépare son exécution.	<code>main()</code>
<code>runner.py</code>	Orchestre les différents modes d'exécution du simulateur.	<code>run_train()</code> , <code>run_validation()</code> , <code>run_evaluation()</code> , <code>run_animation()</code> , <code>run_demo()</code>

5.5 Répertoire des dossiers générés

En complément des fichiers sources, le projet contient plusieurs dossiers créés au cours des différents modes d'exécution. Le Tableau 22 suivant récapitule leur organisation et le type de résultats qu'ils contiennent.

TABLE 22 – Répertoire des dossiers générés.

<i>Fichier</i>	<i>Contenu</i>
<code>./logs/</code>	
<code>train/<policy>/</code>	Métriques et informations de débogage générées durant l'entraînement.
<code>validation/<policy>/<checkpoint>/</code>	Résultats de validation associés à une politique et à un checkpoint.
<code>evaluation/<policy>/<checkpoint>/</code>	Résultats d'évaluation associés à une politique et à un checkpoint.
<code>./figures/</code>	
<code>train/<policy>/</code>	Figures générées durant l'entraînement.
<code>validation/<policy>/<checkpoint>/</code>	Figures et animations produites durant la validation.
<code>evaluation/<policy>/<checkpoint>/</code>	Figures et animations produites durant l'évaluation.
<code>demo/<policy>/<checkpoint>/</code>	Visualisations générées avec le mode <code>animate</code> et utilisées par le mode <code>demo</code> .
<code>./rl/sac/</code>	
<code>weights/<policy>/<checkpoint>/</code>	Poids des réseaux correspondant à un état sauvegardé d'une politique.

5.6 Répertoire des principales classes

Les principales classes du projet sont répertoriées dans le Tableau 23 ci-dessous afin de permettre d'identifier rapidement leur rôle et le module dans lequel elles sont définies.

TABLE 23 – Répertoire des principales classes.

<i>Classe</i>	<i>Rôle</i>
./simulator/entities/	
Entity	Classe de base commune aux entités du simulateur.
StaticEntity	Représente les obstacles statiques et les murs.
MovingEntity	Représente les entités mobiles et les obstacles dynamiques.
Agent	Représente un robot mobile utilisant la politique de navigation.
./simulator/environment/	
Environment	Gère l’environnement, les observations, les récompenses et l’évolution d’un épisode.
Manager	Génère et initialise les scénarios et leurs entités.
GridMap	Fournit la représentation discrète utilisée pour les observations et le calcul des trajectoires.
AStar	Calcule les trajectoires des obstacles dynamiques sur la grille.
./rl/sac/	
SACAgent	Implémente l’agent d’apprentissage SAC partagé par les robots participant à l’apprentissage.
ReplayBuffer	Stocke et échantillonne les transitions utilisées pendant l’apprentissage.
FeatureExtractor	Encode les observations en une représentation latente.
ActorNetwork	Implémente le réseau actor de la politique SAC.
CriticNetwork	Implémente les réseaux critic de SAC.
./configs/config.py	
Task	Regroupe les références aux configurations constituant une tâche.
EnvConfig	Définit les paramètres d’un environnement.
AgentConfig	Définit les paramètres d’un agent.
RewardConfig	Définit les paramètres de la fonction de récompense.
SACConfig	Regroupe les paramètres et hyperparamètres de SAC.
Curriculum	Regroupe les paramètres du curriculum probabiliste.

FAQ

Cette section répond aux principales questions pouvant survenir lors de l'utilisation du simulateur. Elle complète les sections précédentes en regroupant les procédures courantes ainsi que certaines précisions concernant l'entraînement, la validation, l'évaluation et la gestion des résultats.

Comment entraîner une nouvelle politique ?

Pour entraîner une nouvelle politique, il faut d'abord définir une section de tâches dans le fichier `tasks.yaml`. Chaque tâche référence une configuration d'environnement, une configuration d'agent et une fonction de récompense définies dans les fichiers YAML correspondants.

L'entraînement est ensuite lancé à l'aide de la sous-commande `train`, en précisant la section de tâches et le nom de la politique à créer :

```
1 python main.py train --task-section "train_v1" --policy-name "policy_v1"
```

Par défaut, la commande exécute l'entraînement selon un curriculum probabiliste, mais le curriculum séquentiel peut être activé avec l'option `--sequential-curriculum`. Il est également possible de ne sélectionner qu'une partie des tâches à l'aide de `--start-task` et `--end-task`.

Un entraînement peut également être initialisé à partir d'un checkpoint existant à l'aide des options `--init-policy-name` et `--init-checkpoint-name`.

Quelle est la différence entre la validation et l'évaluation ?

La validation consiste à mesurer les performances d'une politique sur les tâches utilisées durant son apprentissage. Elle permet notamment d'étudier les performances acquises au cours du curriculum et, dans le cas d'un curriculum séquentiel, de comparer les checkpoints obtenus à différentes étapes. L'évaluation utilise au contraire des tâches dédiées, différentes de celles rencontrées durant l'apprentissage, afin d'étudier les performances finales de la politique et sa capacité de généralisation.

Le simulateur ne vérifie pas automatiquement la nature de la section de tâches fournie. Il appartient donc à l'utilisateur de sélectionner une section cohérente avec le mode utilisé.

Quel checkpoint utiliser ?

Le checkpoint à utiliser dépend de l'objectif recherché. Le checkpoint `final` correspond à l'état de la politique à la fin de l'entraînement. Dans le cas du curriculum probabiliste, le checkpoint `best` conserve l'état ayant obtenu les meilleures performances au cours du curriculum, tandis que `current` correspond au dernier état sauvegardé de la politique en cours d'apprentissage. Dans le cas d'un curriculum séquentiel, des checkpoints portant le nom des différentes tâches sont également sauvegardés. La stratégie de validation `matching` permet alors de charger automatiquement, pour chaque tâche, le checkpoint portant le même nom.

Il est possible que la condition d'arrêt du curriculum ne soit jamais atteinte et que l'utilisateur décide

d'interrompre l'apprentissage avant son terme, notamment dans le cas du curriculum probabiliste. Le checkpoint final n'est alors pas sauvegardé. Dans cette situation, il est préférable d'utiliser le checkpoint best pour la validation ou l'évaluation de la politique.

Quelle est la différence entre `nb_agents` et `--n-trained-agents` ?

Le paramètre `nb_agents`, défini dans la configuration de l'environnement, indique le nombre total de robots présents dans le scénario. L'option `--n-trained-agents` indique quant à elle combien de ces robots participent directement à l'apprentissage.

Dans le cadre du *parameter sharing*, les transitions des agents sélectionnés alimentent un même *replay buffer* et contribuent à la mise à jour d'une politique SAC commune. Les autres agents présents dans le scénario utilisent la politique courante sans contribuer directement à son apprentissage.

Comment reprendre un entraînement ?

Un nouvel entraînement peut être initialisé à partir d'un checkpoint existant à l'aide des options `--init-policy-name` et `--init-checkpoint-name`, qui doivent être renseignées conjointement.

Par exemple, le checkpoint current d'une politique entraînée avec le curriculum probabiliste peut être utilisé pour poursuivre l'apprentissage à partir du dernier état sauvegardé des réseaux. Le nom fourni avec `--policy-name` détermine le nom sous lequel le nouvel entraînement sera enregistré.

La reprise s'effectue à partir des poids sauvegardés dans le checkpoint, les autres éléments internes de l'entraînement qui ne sont pas sauvegardés, tel que le *replay buffer*, sont réinitialisés.

Pourquoi le nombre d'obstacles n'est-il pas toujours interprété de la même manière ?

L'interprétation des paramètres liés aux obstacles dépend du mode de génération sélectionné. Dans un environnement random, ils déterminent directement le nombre d'obstacles à générer, sous réserve des contraintes de placement. Dans les modes structurés tels que `warehouse`, `fixed_n`, `crossing` ou `hospital`, certains paramètres sont également utilisés pour déterminer automatiquement la géométrie et l'organisation du scénario.

Il est donc recommandé de consulter la description de `env_mode` et `agent_mode` dans la section consacrée à la configuration des environnements afin de connaître le comportement associé à chaque mode.

Comment ajouter un nouveau scénario ?

Pour créer un nouveau scénario à partir des modes de génération existants, il suffit d'ajouter une nouvelle configuration dans `environments.yaml`, puis de créer une tâche dans la section souhaitée de `tasks.yaml`. Cette tâche doit référencer la configuration d'environnement ainsi qu'une configuration d'agent et une fonction de récompense existantes ou nouvellement définies.

Si le scénario nécessite une nouvelle logique de génération qui ne peut pas être représentée par

les modes `env_mode` ou `agent_mode` existants, la classe `Manager` doit également être étendue afin d'implémenter les nouvelles règles de génération.

Quelle est la différence entre `animate` et `demo` ?

La sous-commande `animate` exécute une politique et génère les visualisations correspondantes, notamment les animations GIF et MP4. La sous-commande `demo` charge quant à elle une animation précédemment générée et permet de la parcourir de manière interactive. Il est donc nécessaire d'avoir exécuté `animate` au préalable avec la même section de tâches, la même politique et le même checkpoint.

Où sont enregistrés les résultats ?

Les métriques sont enregistrées dans `logs/`, les figures et animations dans `figures/`, et les poids des politiques dans `rl/sac/weights/`. Les résultats sont ensuite organisés selon le mode d'exécution, la politique et, lorsque cela est pertinent, le checkpoint utilisé. L'organisation détaillée de ces dossiers est présentée dans la Section [5](#).

Table des figures

1	Architecture fonctionnelle du simulateur.	5
2	Diagramme de classes simplifié du simulateur.	6
3	Workflow d'exécution et organisation des résultats générés par le simulateur.	19

Liste des tableaux

1	Description des principaux dossiers et fichiers du projet.	4
2	Description des fichiers de configuration.	8
3	Description des paramètres de Task.	9
4	Description des paramètres de EnvConfig.	10
5	Description des modes de génération de <code>env_mode</code>	11
6	Description des modes de génération de <code>agent_mode</code>	11
7	Description des paramètres de AgentConfig.	13
8	Description des paramètres de RewardConfig.	14
9	Description des paramètres de SACConfig.	15
10	Description des paramètres de Curriculum.	16
11	Description des modes d'exécution.	18
12	Options de la sous-commande <code>train</code>	20
13	Options de la sous-commande <code>validate</code>	22
14	Options de la sous-commande <code>evaluate</code>	24
15	Options de la sous-commande <code>animate</code>	26
16	Options de la sous-commande <code>demo</code>	27
17	Fichiers générés dans le dossier <code>logs/</code>	29
18	Fichiers générés dans le dossier <code>figures/</code> en mode <code>train</code>	29
19	Fichiers générés dans le dossier <code>figures/</code> en mode <code>validate</code> ou <code>evaluate</code>	30
20	Fichiers générés dans le dossier <code>rl/sac/weights/</code>	30
21	Répertoire des fichiers sources.	32
22	Répertoire des dossiers générés.	34
23	Répertoire des principales classes.	35